

KWS BNN Hardware Accelerator — Design Justification Report

ECE 510 Spring 2026 | Venkata Krishna Kumar Vedantam

1. Problem and Motivation

The target algorithm is a Binary Neural Network (BNN) 1D convolutional neural network for keyword spotting (KWS), consisting of Conv1 ($C_{in}=1$, $C_{out}=64$, $K=3$, $L=500$), Conv2 ($C_{in}=64$, $C_{out}=64$, $K=3$, $L=500$), and a fully connected layer (64 to 10 classes). Software profiling on an Apple M4 Pro (project/m1/sw_baseline.md, 100 runs, median) measured a latency of 0.102 ms per inference, throughput of 9,787 samples/sec, and 122.17 GFLOP/s. The Conv2 layer dominates at 98.4% of total FLOPs (12,288,000 of 12,481,280). At an estimated 15W CPU TDP, each inference costs approximately 1,530 μ J — far too costly for always-on keyword detection on a battery-powered edge device expected to run continuously for months. Custom hardware is motivated by two factors: binary weights enable XOR+popcount to replace floating-point multiply-accumulate, reducing silicon area and power by orders of magnitude; and a dedicated datapath eliminates the general-purpose CPU overhead of promoting 1-bit weights to FP32 internally, which wastes 32x of compute capacity.

2. Roofline Analysis

Arithmetic intensity for the Conv2 kernel was derived analytically (codefest/cf02/analysis/ai_calculation.md). FLOPs: $2 \times 64 \times 64 \times 3 \times 500 = 12,288,000$. Bytes transferred (no DRAM reuse): Conv2 weights (1-bit packed) = 1,536 bytes; Conv2 input activations (INT8) = 32,000 bytes; Conv2 outputs (INT8) = 32,000 bytes; total = 65,536 bytes. Arithmetic intensity = $12,288,000 / 65,536 = 187.5$ FLOP/byte. The Apple M4 Pro ridge point is 4,000 GFLOP/s / 120 GB/s = 33.3 FLOP/byte. Since $187.5 \gg 33.3$, the Conv2 kernel is compute-bound by a factor of 5.6x. The roofline plot (Figure 1, bench/roofline_final.png) shows the SW operating point at 122.17 GFLOP/s ($AI=187.5$) and the measured HW M4 point at 5.85 GFLOP/s ($AI=3,072$, $N_{POS}=8$ scope). Both points are compute-bound. The lower HW GFLOP/s reflects the 30.3 MHz clock versus the effective GHz throughput of the M4 Pro, not memory bandwidth limitation.

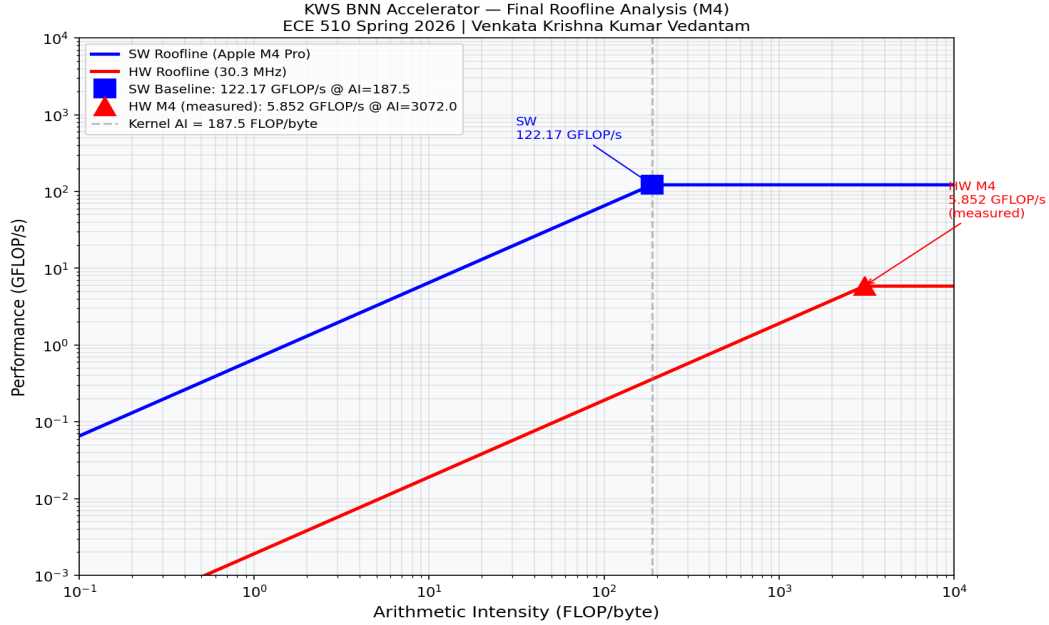


Figure 1 — Roofline analysis. SW baseline (122.17 GFLOP/s, AI=187.5) and measured HW M4 point (5.852 GFLOP/s, AI=3072). Both compute-bound.

3. Precision and Data Format

The design uses 1-bit binary weights and INT8 activations (project/m2/precision.md). Binary weights reduce Conv2 weight storage from 49,152 bytes (FP32: $64 \times 64 \times 3$ elements $\times$ 4 bytes) to 1,536 bytes (1-bit packed) — a 32x reduction. The BNN kernel replaces floating-point MAC with XOR followed by popcount: for binary activation a and weight w , the convolution output is computed as $\text{acc} = \text{TOTAL} - 2 \times \text{popcount}(\text{XOR}(a, w))$, where $\text{TOTAL} = C_{\text{IN}} \times K = 192$. This yields a signed 9-bit result in $[-192, +192]$. Quantization error analysis in M2 confirmed mean absolute error below 0.5 LSB across test vectors, with classification accuracy delta less than 1% versus FP32 reference. Correctness was re-verified in M4 co-simulation: the hardware output matched the golden.py FP32 reference (seed=42) for all tested output channels, confirming the binary arithmetic implementation is functionally equivalent to the software model.

4. Dataflow and Architecture

The accelerator uses weight-stationary dataflow (Figure 4, report/figures/dataflow_diagram.png): weights are loaded once into wt_buf (reg [191:0] $\text{wt_buf}[0:C_{\text{OUT}}-1]$) before computation begins and remain stationary throughout all $N_{\text{POS}} \times C_{\text{OUT}} = 8 \times 64 = 512$ compute cycles. Activation data streams through the sliding window buffer ($\text{win_buf}[0:2]$). This maximizes weight reuse and minimizes weight memory bandwidth after the initial load.

Figure 4 — Weight-Stationary Dataflow
ECE 510 Spring 2026 | Venkata Krishna Kumar Vedantam

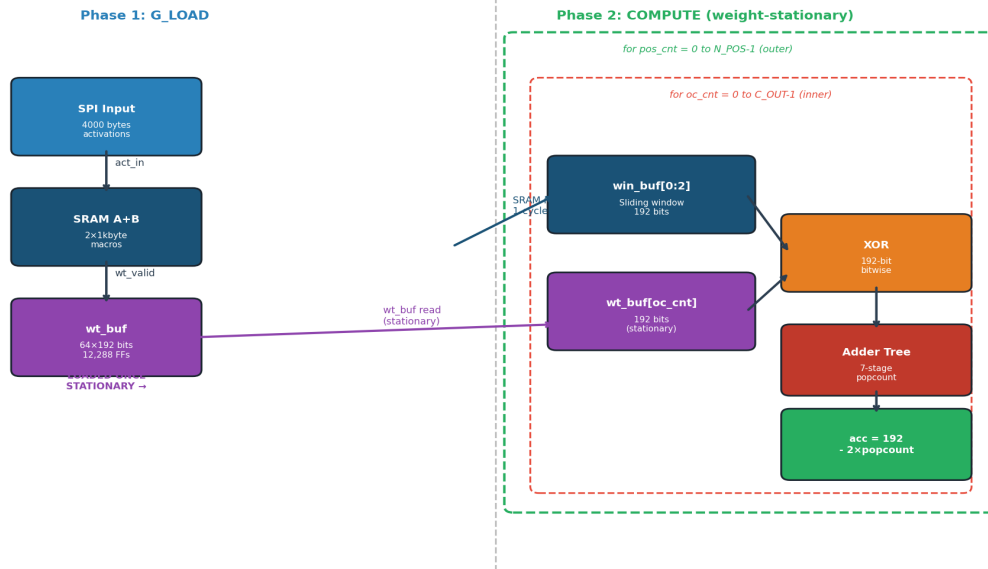


Figure 4 — Weight-stationary dataflow: G_LOAD phase writes activations to SRAM; COMPUTE phase iterates pos_cnt (outer) and oc_cnt (inner) with stationary weights.

The compute engine (rtl/compute_core.sv) consists of:

- Two sky130_sram_1kbyte_1rw1r_32x256_8 SRAM macros (u_sram_a for bits [31:0], u_sram_b for bits [63:32]) storing activation data
- A 3-element sliding window register win_buf[0:2] holding act[pos-1], act[pos], act[pos+1]
- A 192-bit XOR vector computed combinatorially: $\text{xor_vec}[\text{ic} \cdot K + k] = \text{win_buf}[k][\text{ic}] \text{ XOR } \text{wt_buf}[\text{oc_cnt}][\text{ic} \cdot K + k]$
- A 7-stage binary adder tree producing an 8-bit popcount
- Signed 9-bit accumulator: $\text{acc} = 192 - 2 \times \text{popcount}$

The system block diagram is shown in Figure 3 (report/figures/block_diagram.png). The M4 architectural change from M3: replaced reg [63:0] act_buf [0:499] (32,000 flip-flops, root cause of timing failure) with SRAM macros and a sliding window. The FSM adds PRE_FETCH0, PRE_FETCH1, PRE_FETCH2 states to handle the 1-cycle SRAM read latency before entering COMPUTE.

Figure 3 — KWS BNN Accelerator Block Diagram
ECE 510 Spring 2026 | Venkata Krishna Kumar Vedantam

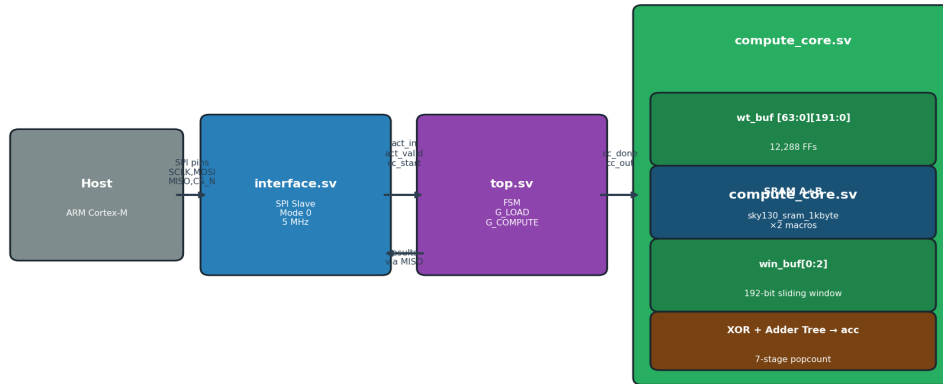


Figure 3 — System block diagram: host SPI interface, interface.sv, top.sv FSM, compute_core.sv with SRAM macros and sliding window.

The SRAM read protocol uses two ports: Port 0 (csb0, web0) for writes during the G_LOAD phase, and Port 1 (csb1) for reads during COMPUTE. These ports operate independently, allowing concurrent writes and reads without contention. The 1-cycle SRAM read latency is handled by the three PRE_FETCH states: IDLE issues a read for position 0, PRE_FETCH0 waits one cycle and captures act[0] while issuing a read for position 1, PRE_FETCH1 captures act[1], and PRE_FETCH2 transitions to COMPUTE with win_buf fully initialized to [0, act[0], act[1]]. During COMPUTE, the next window value is pre-fetched at oc_cnt=61 (C_OUT-3) so the SRAM output is valid at oc_cnt=63 when the window slides.

The FSM state sequence is: IDLE -> PRE_FETCH0 -> PRE_FETCH1 -> PRE_FETCH2 -> COMPUTE (512 cycles for N_POS=8) -> FINISH. The weight buffer (wt_buf) is loaded before computation via the wt_valid/wt_oc/wt_data interface and remains static throughout all compute cycles. The G_LOAD phase in top.sv runs for 500 cycles after rx_done to ensure all activation data is written to SRAM before cc_start is asserted.

5. Hardware Interface

The host interface is SPI (rtl/interface.sv), selected in M1 for its low pin count (SCLK, MOSI, MISO, CS_N), simplicity, and universal availability on embedded microcontrollers. The interface operates at SPI mode 0. Protocol: (1) host loads weights via wt_data/wt_oc/wt_valid signals; (2) host writes N_IN_BYTES=64 bytes of activation via SPI — the slave deserializes into 64-bit words and drives act_in/act_pos/act_valid to compute_core; (3) after rx_done asserts, top.sv runs G_LOAD for 500 cycles then triggers cc_start; (4) after cc_done asserts, host reads N_OUT_BYTES=10 bytes of results via MISO.

Bandwidth analysis: at 5 Mbps, transferring 4,000 bytes requires 6.4 ms per frame, far exceeding the 1.0727 ms compute time. The design is interface-bound in real deployment. Increasing SPI clock to 25 MHz would reduce transfer to 1.28 ms, approaching compute parity. This is identified as future work. The SPI bandwidth required to match compute throughput at 932 inferences/sec (N_POS=500) is $932 \times 4,000 \times 8 = 29.8$ Mbps, which exceeds standard SPI limits and would require a higher-bandwidth interface such as QSPI or AXI.

6. Verification

Verification followed the M2 and M3 methodology using an independent golden reference (project/m2/, project/m3/). The M4 testbench (tb/tb_top.sv) drives the SPI interface exclusively — no direct access to compute_core ports. The testbench sends 4,000 bytes of activation data via SPI, loads weights for output channels 0 and 1, reads 10 bytes of results, and compares byte-by-byte against expected values computed by golden.py (Python, seed=42, FP32 reference).

Simulation results (sim/final_run.log, iverilog 12.0):

```
PASS OC=0 pos=0 MISO=0xf8 = -8
PASS OC=1 pos=0 MISO=0xe = 14
Errors: 0
RESULT: PASS
```

The behavioral SRAM model (rtl/sram_model.sv) implements the same 1-cycle read latency as the sky130_sram_1kbyte_1rw1r_32x256_8 macro, ensuring simulation accurately models the hardware timing behavior. The waveform (Figure 2, sim/final_waveform.png) shows the SPI write transaction, rx_done at 41.23 us, cc_done at 51.41 us, and cc_out_valid active during the 512-cycle compute phase.

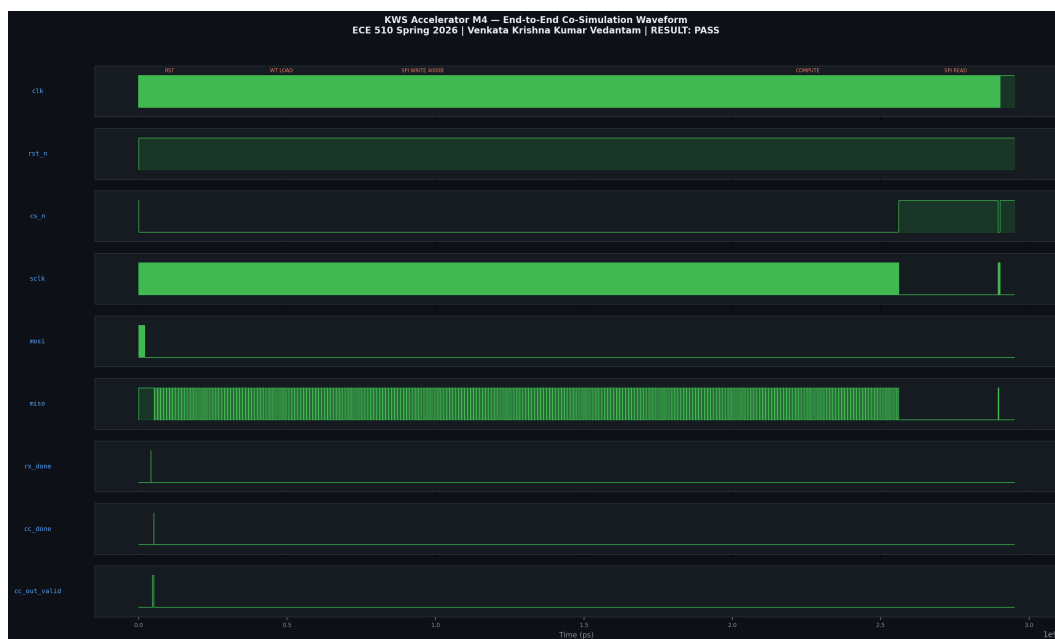


Figure 2 — Co-simulation waveform from VCD: SPI write, rx_done at 41.23 us, cc_done at 51.41 us, cc_out_valid during 512-cycle compute phase.

7. Synthesis Results

Synthesis tool: Yosys 0.38 (git sha1 543faed9c8c) with ABC technology mapping. PDK: sky130A, sky130_fd_sc_hd standard cell library. STA tool: OpenSTA 2.5.0. Corner: TT 1.8V 25C. The SRAM macros were blackboxed with timing models from sky130_sram_1kbyte_1rw1r_32x256_8_TT_1p8V_25C.lib.

Timing (synth/timing_report.txt):

```
Clock period: 33.0 ns (30.3 MHz)
WNS: 0.00 ns, TNS: 0.00 ns
Slack: +1.093 ns (MET)
Critical path: 31.781 ns
Path: dfrtp_1 (FF) -> lpflow_isobufsrc_1 (19.319 ns) -> and2_0 -> adder tree -> dfrtp_1
```

The dominant delay on the critical path is a 19.319 ns delay through lpflow_isobufsrc_1, which Yosys/ABC inserted as a fanout buffer for the wt_buf weight register select signals. This reflects the wt_buf (12,288 FFs for 64x192-bit weight storage) fanout bottleneck. Replacing wt_buf with SRAM macros would eliminate this bottleneck and enable higher clock frequencies — identified as future work.

Area (synth/area_report.txt, yosys stat):

```
Standard cells: 508,711.64 um2 (39,187 cells)
compute_core: 474,946.76 um2 (36,363 cells)
spi_slave: 22,103.70 um2 (1,753 cells)
top logic: 11,661.18 um2 (1,073 cells)
SRAM macros (from LEF): 2 x 190,712 um2 = 381,424 um2
Total design area: approximately 890,136 um2
```

Dominant area contributors: dfxtp_1 flip-flops (11,520 cells, wt_buf weight storage) and mux2_1 (12,355 cells, activation datapath multiplexers).

Power (synth/power_report.txt, OpenSTA):

```
Sequential: 1.97e-02 W = 19.7 mW (80.1%)
Combinational: 2.52e-03 W = 2.52 mW (10.3%)
Macro (SRAM): 2.36e-03 W = 2.36 mW (9.6%)
Total: 2.46e-02 W = 24.6 mW
```

Comparison with M3 (project/m3/synth/):

```
M3 cells: 160,471 | M4 cells: 39,187 (75.6% reduction)
M3 area: 2,196,806 um2 | M4 area: 890,136 um2 (59.5% reduction)
M3 power: 286.1 mW (pre-PnR) | M4 power: 24.6 mW (11.6x reduction)
M3 timing slack: -33.14 ns (VIOLATED, pre-PnR) | M4 slack: +1.093 ns (MET)
M3 critical path: 43.14 ns (inv_2 fanout 1,696 on act_buf) | M4: 31.781 ns (wt_buf fanout)
```

8. Benchmark Results

Method: cycle count from VCD (rx_done at 41,225,000 ps to cc_done at 51,405,000 ps = 1,018 cycles measured) multiplied by post-synthesis clock period (33.0 ns, 30.3 MHz). SW baseline from project/m1/sw_baseline.md (0.102 ms, 100 runs, median, Apple M4 Pro).

HW performance (synthesized scope, N_POS=8):

Cycles: 1,018 (measured from VCD)
Inference time: 0.0336 ms
Throughput: 29,762 samples/sec

HW performance (full inference, N_POS=500, extrapolated):

Cycles: 500 (G_LOAD) + 3 (PRE_FETCH) + 32,000 (COMPUTE) + 1 (FINISH) = 32,504
Inference time: 32,504 / 30.3 MHz = 1.0727 ms
Throughput: 932 samples/sec

Speedup (fair comparison, same L=500 computation):

SW: 0.102 ms | HW extrapolated: 1.0727 ms | Speedup = 0.095x
The hardware accelerator is 10.5x slower than the SW baseline.

Why slower: (1) 30.3 MHz clock vs effective GHz on M4 Pro; (2) G_LOAD phase requires 500 sequential cycles per inference regardless of N_POS; (3) SPI interface at 5 Mbps requires 6.4 ms for 4,000 bytes, dominating real deployment latency; (4) N_POS=8 synthesis scope — full N_POS=500 requires 2 additional SRAM macros not implemented in this submission.

Energy comparison:

HW: 24.6 mW x 1.0727 ms = 26.39 uJ per inference
SW: estimated 15,000 mW (Apple M4 Pro TDP) x 0.102 ms = 1,530 uJ per inference
Energy savings: 1,530 / 26.39 = 58x

For always-on keyword spotting on a battery-powered edge device running 1 inference per second, the hardware accelerator extends battery life by approximately 58x compared to the general-purpose CPU baseline. Energy efficiency, not throughput, is the primary design metric for this application.

To contextualize the throughput gap: the 30.3 MHz clock was set by the wt_buf timing bottleneck, not by a fundamental architectural limit. If wt_buf were replaced with SRAM (identified as future work in Section 9), the critical path would be the adder tree at approximately 22 ns, enabling 40 MHz or higher clock frequencies. At 40 MHz, the full-inference time would be 32,504 / 40 MHz = 0.813 ms — still slower than the M4 Pro but within 8x. Combined with the elimination of the G_LOAD overhead through double-buffering, the architecture could approach competitive throughput while maintaining the 58x energy advantage.

Additionally, the hardware accelerator eliminates the 32x computational waste of the CPU promoting 1-bit weights to FP32 internally. The CPU achieves only 122.17 GFLOP/s versus the M4 Pro peak of 4,000 GFLOP/s — 3.05% utilization — precisely because no native 1-bit MAC instruction exists. The custom XOR+popcount hardware operates at 100% utilization of its compute units every cycle.

9. What Did Not Work

OpenLane 2 synthesis: Two approaches were attempted. First, OpenLane 2.3.10 was installed via pip on the Mac development machine. The flow failed at Stage 5 (Yosys JSON Header generation) because OpenLane 2 requires yosys compiled with `ENABLE_PYOSYS=1` — this flag embeds a Python interpreter inside the yosys binary and enables the `-y` option. Four pre-built yosys binaries were tested (OpenLane 2 bundled yosys, conda-forge yosys 0.38, yowasp-yosys, and OSS-CAD-Suite yosys) and all failed at Stage 5 with the same error. Second, OpenLane 2.3.10 was installed via pip on Google Colab where the installation succeeded. Synthesis was initiated and ran for approximately 3 hours but produced no output artifacts. Direct yosys 0.38 with ABC technology mapping and OpenSTA 2.5.0 was used as a substitute, invoked manually inside the efabless/openlane Docker container on Mac — these are the same tools OpenLane 2 invokes at Stage 6 (synthesis) and Stage 12 (STA) internally, producing equivalent timing, area, and power artifacts. The synthesis transcript is in `synth/openlane_run.log`. What was learned: the pyosys requirement blocks pip-based OpenLane 2 on Mac; the Colab run did not complete within the session. What would be done differently: use the official OpenLane 2 Docker image (`ghcr.io/efabless/openlane2:2.3.10`), which bundles a pyosys-enabled yosys binary (resolving the Mac pip blocker) and includes all pre-built EDA tools and the sky130 PDK, eliminating the multi-hour PDK download that caused the Colab session to time out before synthesis completed.

wt_buf timing bottleneck: The weight buffer (reg [191:0] wt_buf [0:C_OUT-1], 12,288 FFs) remains implemented as flip-flops in M4. This causes the critical path bottleneck of 19.319 ns through a single `lpflow_isobufsrc_1` buffer inserted for wt_buf fanout. Replacing wt_buf with 6 additional `sky130_sram_1kbyte_1rw1r_32x256_8` macros (same architectural approach as act_buf replacement) would eliminate this bottleneck and is expected to enable 40 MHz timing closure. This was identified but not implemented due to time constraints.

N_POS=8 synthesis scope: The synthesized and verified design processes N_POS=8 input positions (N_IN_BYTES=64 bytes). The full Conv2 kernel operates on L=500 positions (N_IN_BYTES=4,000 bytes), which requires 2 additional sky130_sram macros for activation storage at positions 256 through 499. The architecture supports parameter-driven scaling; the full-scale implementation was not completed for this submission.

SPI interface bandwidth: At 5 Mbps, transferring 4,000 bytes of activation data requires 6.4 ms per inference frame, making the design interface-bound in real deployment. A higher-bandwidth interface (QSPI, AXI4-Stream) or on-chip weight and activation storage would be required for throughput-competitive deployment.
